International Journal of Machine Learning and Cybernetics
https://doi.org/10.1007/s13042-020-01104-8

ORIGINAL ARTICLE

A novel learning-based approach for efficient dismantling of networks

Changjun Fan[1] · Li Zeng[1] · Yanghe Feng[1] · Guangquan Cheng[1] · Jincai Huang[1] · Zhong Liu[1]

Received: 18 June 2019 / Accepted: 24 February 2020

Abstract
Dismantling of complex networks is a problem to find a minimal set of nodes in which the removal leaves the network broken into connected components of sub-extensive size. It has a wide spectrum of important applications, including network immunization and network destruction. Due to its NP-hard computational complexity, this problem cannot be solved exactly with polynomial time. Traditional solutions, including manually-designed and considerably sub-optimal heuristic algorithms, and accurate message-passing ones, all suffer from low efficiency in large-scale problems. In this paper, we introduce a simple learning-based approach, CoreGQN, which seeks to train an agent that is able to smartly choose nodes that would accumulate the maximum rewards. CoreGQN is trained by hundreds of thousands self-plays on small synthetic graphs, and can then be able to generalize well on real-world networks across different types with different scales. Extensive experiments demonstrate that CoreGQN performs on par with the state-of-art algorithms at greatly reduced computational costs, suggesting that CoreGQN should be the better choice for practical network dismantling purposes.

Keywords Network dismantling · Graph embedding · Reinforcement learning

1 Introduction

Network dismantling, or network attack, is one of the fundamental structural problem in network science [1]. The problem seeks to choose the minimum nodes, of which the removal would result in the remaining components with sub-extensive size, e.g., 1% of the original size [7]. This problem has very wide and significant practical applications, such as to immunize the epidemic propagation in populations via separating a small subset of critical persons [30], block the rumor spreading on social networks by limiting some highly influential nodes [20], prevent the virus diffusion in computer networks through cutting off some hub hosts [10], disrupt the malicious organization by arresting some leader suspects[12, 14], etc.

Successfully addressing the network dismantling problem is of great significance, however, there are exponential number of candidate solutions as the network size grows, making it impossible to find the optimal one in polynomial time. So far, the network dismantling problem has been mainly addressed by heuristic methods [5, 15, 16, 28, 30, 35], or some message-passing algorithms [7, 29]. The former methods often greedily select target nodes based on local metrics, like node degree, which often leads to sub-optimal solutions; the latter ones are more accurate and global, while they need to iterate certain steps on the whole network to select the suitable candidate nodes [35], which would sacrifice some efficiency. Moreover, the close-to-optimal performances of the latter methods is only theoretically justified on random networks, and cannot be guaranteed on real-world networks with many loops [35].

Recently, many researches [7, 29, 35] find the correlation between decycling and dismantling problems. Network decycling problems [36] seeks to remove as few nodes as possible such that the remaining graph contains no loops. Many results have shown that the decycling is closely related to dismantling, the best dismantling result is often achieved by first finding a decycling solution and then re-inserting some nodes to close short loops while do not increase too much the size of the largest component. Previous efforts on network decycling either adopt message passing [7] or belief propagation [29] procedures, which often have to update

Changjun Fan and Li Zeng these authors have contributed equally to this work.

✉ Yanghe Feng
fengyanghe@nudt.edu.cn

[1] College of Systems Engineering, National University of Defense Technology, Changsha 410000, People's Republic of China

Published online: 12 March 2020

many iterations on the whole network, or just take the simple greedy methods, like removing the highest degree node at each adaptive step [35], which may lead to the sub-optimal. Instead, we purpose **CoreGQN**, (k-**Core G**raph **Q N**etwork), a learning based framework to handle the network dismantling problem. We treat the network decycling as a Markov Decision Process (MDP) on networks: the agent interacts with the environment through a sequence of states, actions, and rewards. Here, the *environment* is the network to decycle, the *state* is defined as the 2-core of the remaining network, the *action* is the target node to remove, the *reward* is defined as -1. The agent is off-line trained by self-playing on hundreds of thousands small toy networks, e.g., generated from the Barabási-Albert (BA) [4] model. In the inference stage, the well-trained agent can be applied to decycle various real-world networks across different types. After all cycles are broken, the network degrades into a forest, we break them into small components by removing a fraction of root nodes that vanishes in the large size limit [7, 29, 35]. In the end, to reduce the number of removed nodes, we greedily reinsert some nodes that close short cycles without exceeding the large size limit.

We conduct extensive experiments on nine real-world networks of five different types, with up to millions of nodes and tens of millions of edges. The results show that CoreGQN achieves at least on par with the current best performance while is orders of magnitude faster.

The main contributions of this paper are summarized as follows:

1. We transform the network decycling into a learning problem, where an agent that learns to decycle the network with the minimum nodes is trained on small toy networks, and can be applied on various real-world networks.
2. We propose a novel learning-based framework, CoreGQN, for efficient dismantling of networks. The model first trains an agent to decycle the network, then greedily breaks the remaining acyclic graph by removing the root nodes. Finally reinserts some nodes to close short cycles without exceeding the large size limit.
3. We perform extensive experiments on real-world datasets in different scales and in different domains. Our results demonstrate that CoreGQN performs on par with or better than state-of-the-art baselines, and is orders of magnitude faster.

The rest of the paper is organized as follows. We systematically review related work in Sect. 2. After that, we introduce the architecture of CoreGQN in detail in Sect. 3. Section 4 presents the evaluation of our model on real-world networks. Finally, we conclude the paper in Sect. 5.

2 Related work

2.1 Network dismantling

Network dismantling problem is defined as to remove the minimal number of nodes, so as the removal of which leads to a remaining giant connected component (GCC) with the sub-extensive size, e.g., 1% of the original size. It can be formalized as the following:

$$\min\{q : s(q) = 0.01\} \tag{1}$$

where q is the fraction of nodes to be removed, $s(q)$ is the size of the remaining GCC after removing q fraction of nodes.

There have been plenty of work towards the network dismantling problem. Most straightforward ones are to sequentially target at important nodes measured by some local/global centrality metrics, such as degree [30], betweenness [16], closeness [5], pagerank [8], etc. Morone et al [28] introduce a new local centrality metric: collective influence, which is the product of node's reduced degree (degree minus one) and the sum of reduced degrees from its neighbors d-hops away. All these heuristics either suffer from sub-optimal performances, or become computationally prohibitive in large networks. Recently, there are another line of attempts which connects decycling and dismantling together [7, 29, 35]. Braunstein [7] first accurately described the connection between decycling and dismantling, and proves that almost the same set of nodes is needed to achieve the both problems. Furthermore, Braunstein put forward a three-stage procedure for efficient network dismantling, which first used Min-Sum message passing [2, 3] for decycling, then applied the tree breaking on the remaining forest after all cycles are broken, finally to reduce the total number of removed nodes, it reinserted some nodes that close short cycles without increasing too much the size of the largest component. The authors of [29] and [35] share the similar three stages as [7], only differs in the decycling part. Zdeborov et al. [35] utilized a related but different message-passing algorithm [36], and its performance was very close to [7]. Zdeborov et al. [35] proposed a very simple greedy method for decycling, which recursively removes the highest degree nodes from the 2-core of the network, based on the observation that a graph contains no cycles if and only if its 2-core is empty.

Empirically, the decycling-driven dismantling methods performs consistently better than heuristic ones on real-world networks, and for the former methods, decycling plays the core role. Current decycling algorithms either need to iterate certain rounds on the whole network that takes longer time, or simply utilize the greedy heuristic which encounters the sub-optimal.

2.2 Machine learning for combinatorial optimization

Network dismantling is a typical combinatorial optimization problem. Recently, there emerges some machine learning models to tackle the traditional combinatorial optimization problems. Bello [6] presented a framework to tackle the travelling salesman problem (TSP), by using neural networks and reinforcement learning. They trained a recurrent network that, given a set of city coordinates, predicted a distribution over different city permutations, and used policy gradient to update the parameters. Li et al. [25] presented a learning-based approach to compute solutions for graph combinatorial optimization problems. They combined graph convolutional network that was trained to estimate the likelihood of whether one node is part of the optimal solution, and useful algorithmic elements from classic heuristics to achieve the current best results. Khalil [21] combined the structure2vec, a graph embedding model to represent the graph, and reinforcement learning, to learn a greedy policy that behaves like a meta-algorithm that incrementally constructs a solution. They demonstrated its effectiveness on a diverse range of optimization problems over graphs, e.g., Minimum Vertex Cover, Maximum Cut and Traveling Salesman problems.

Recently, Fan [13] purposed a deep learning architecture to solve the network dismantling problem, however, they seek to minimize the area under the robustness curve, a totally different goal as this paper. Besides this work, there seems no other research that applied machine learning techniques to solve the network dismantling problem.

3 CoreGQN model

In this section, we introduce the proposed framework, namely CoreGQN, for efficient network dismantling. We first introduce the architecture of CoreGQN in detail, then describe the training process, and ends in analyzing the time complexity of the model.

3.1 Architecture of CoreGQN

Figure 1 illustrates an overview of CoreGQN framework. The framework mainly contains three stages to dismantle a network. (i) *Decycle stage*. This stage seeks to remove the minimum fraction of nodes to break all the loops. We adopt a deep architecture to achieve the goal. First use graph embedding techniques to represent the nodes in the 2-core, then calculate the Q-values for these nodes with their representations, finally greedily remove those highest Q nodes until there are no cycles; (ii) *Tree break stage*. After all cycles are removed, the network remains to be a forest, we break them into small components with sub-extensive size, e.g., 1% of the original size, by sequentially removing the root nodes. (iii) *Reinsert stage*. In the end, to reduce the total number of removed nodes, we greedily reinsert some nodes that close cycles without increasing too much the size of the largest component. In the following, we describe each stage in detail.

Decycle stage We utilize a fundamentally different way to decycle the network. We first use the graph embedding technique [11, 17, 26, 34] to represent the graph, including both nodes and the whole graph. This graph embedding computes a p-dimensional feature vectors z_v for each node $v \in V_{2-\text{core}}$, where $V_{2-\text{core}}$ indicates all nodes in the 2-core of the current network. More specifically, it recursively aggregates nodes' features according to the 2-core's graph topology, where node features are initialized as constant vectors in this paper. After a few steps of recursions, it will produce an embedding vector for each node, which considers the node's location information within the graph topology. Actually, more update iterations means more distant neighbors' information will be aggregated non-linearly. If one terminates after T iterations, each node embedding z_v will contain information of its $T-$hop neighborhood as determined by the 2-core graph topology. More details about the graph embedding algorithm see Algorithm 1.

Algorithm 1 Graph Embedding

Input: 2-core of the original graph, $G(V, E)$, input features $\{X_v \in \mathbf{R}^{1\times c}, \forall v \in V\}$, terminate iterations T, weight parameters $\theta_1 \in \mathbf{R}^{c\times p}, \theta_2 \in \mathbf{R}^{p\times(p/2)}, \theta_3 \in \mathbf{R}^{p\times(p/2)}$
Output: Embedding vectors $z_v, \forall v \in V$
1: Initialize $h_v^0 \leftarrow ReLU(X_v \cdot \theta_1), h_v^0 \leftarrow h_v^0/\|h_v^0\|_2, \forall v \in V$
2: **for** $k = 1$ to T **do**
3: **for** $v \in V$ **do**
4: $h_v^k \leftarrow ReLU(CONCAT(\theta_2 \cdot h_v^{k-1}, \theta_3 \cdot \sum_{j\in N(v)} h_j^{k-1}))$
5: **end for**
6: $h_v^k \leftarrow h_v^k/\|h_v^k\|_2, \forall v \in V$
7: **end for**
8: $z_v \leftarrow h_v^T, \forall v \in V$

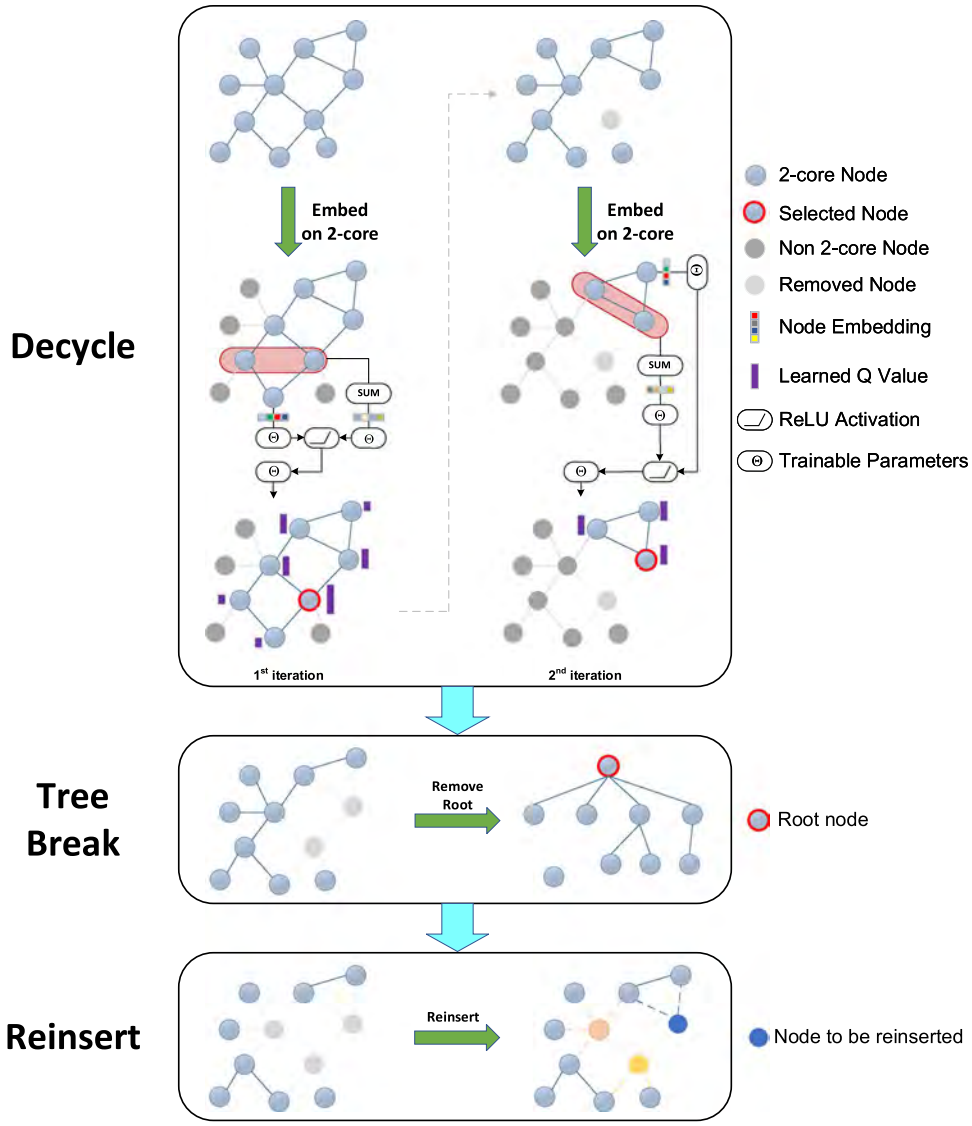

Fig. 1 CoreGQN framework overview

Based on the obtained embeddings of each node, a Q-function is learned to map from state-action pair (s, a) to a scalar value R to represent the long-term gain after taking the current action a. The Q function is parameterized with the MLPs employed with rectified linear unit (ReLU) ($\mathrm{ReLU}(z) = z$ if $z > 0$ and 0 otherwise) activation. More specifically, the current state s is represented by combining the embedding $z_a^\top$ for action (node) a and the sum pooled embedding over the entire graph, and the Q score for the state-action pair (s, a) is computed via Eq. 2:

$$Q(s,a) = \theta_6^\top \mathrm{ReLU}([\theta_4 \sum_{u \setminus V} z_u^\top, \theta_5 z_a^\top]) \tag{2}$$

Here $\theta_4, \theta_5 \in R^{p\times 1}$, $\theta_6 \in R^{p\times 1}$, and $[\cdot,\cdot]$ is the concatenation operator. The embeddings are the final output vectors via the graph embedding, the number of iterations T is usually small, such as $T = 5$. For more efficient computations, we use an architecture in which there is a separate output unit for each possible action, and feed only the state embeddings as the input, so as to compute Q values for all possible actions under a given state with only a single forward pass through the neural networks [27].

Then, we simply remove the highest Q score node, if there are multiple ties, among them one will be randomly chosen. After deleting the node, we need to update the 2-core, which requires on average $O(1)$ operations on sparse networks [35]. Repeat the above graph embedding, Q value calculation and greedy selection steps on the new 2-core, until the 2-core is empty, which means there are no cycles in the remaining graph.

Tree break stage Trees can always be dismantled by a vanishing fraction of nodes [19]. Here we introduce a simple but effective greedy tree breaking strategy [7, 29] that achieves almost the same performance as the optimal algorithm. First, we consider all the leaf nodes (of degree one) of tree T. For

each leaf node i, we let this leaf node to send a message $m_{i\to j=1}$ to its unique neighbor j in the tree T. After considering all these leaf nodes, we remove them and obtain a reduced tree T', and we then consider all the leaf nodes in T'. For each leaf node j in T', we let it send a message $m_{j\to k} \leftarrow 1 + \sum_{i\in N(j)\setminus k}$ to its unique neighbor k in tree T', where $N(j)$ denotes the neighbor nodes of j in the original tree T. It is then obvious that once deleting node j from T, the component sizes of the remaining forest will form the following merged set: $\{m_{l\to j}|l \in N(j) \setminus k\} \cup \{n - m_{j\to k}\}$. After all leaves of T' have been examined, we again delete them to obtain a further reduced tree T'', we repeat to consider all the leaf nodes of T'' in the same way. After a few iterations, all nodes in the original tree T will be checked, and we will be able to identify the optimal node i to break up the tree. After removing node i, repeat the above process until the original tree is broken into pieces with the expected sub-extensive size.

Reinsert stage After the above two stages, a network has been dismantled into small pieces, where the largest component size does not exceed the limit size. However, we can still reduce the total number of removed nodes by adding back some nodes that simply close the short cycles while do not increase the largest component size, this is the well-known reinsertion refinement technique. The reinsertion technique starts from the percolation point, at which the above two phases are over, it adds back in the network one of the removed nodes with the minimum score σ. When the node is reinserted, restore the edges with its neighbors which are in the network (but not the ones with neighbors not yet reinserted, if any). Repeat the above procedure until the current largest component exceeds the limit size.

The performance of the reinsert algorithm depends on the definition of σ_i, which considers all neighboring connected components of node i. Intuitively, σ_i can be defined as the size of the connected component build by restoring node i to the remaining graph, namely,

$$\sigma_i = \sum_j \mathcal{M}_{i,j} + 1 \tag{3}$$

where $\mathcal{M}_{i,j}$ is size of the j-th largest neighboring connected component of node i.

Another commonly adopted measure is defined as:

$$\sigma_i = \begin{cases} \mathcal{N}_i, & \text{if } \operatorname{argmin}_i \mathcal{N}_i \text{ is unique} \\ \mathcal{N}_i + \epsilon\mathcal{M}_{i,2}, & \text{else} \end{cases} \tag{4}$$

where $\mathcal{N}_i$ is the number of neighboring connected components of node i, $\mathcal{M}_{i,2}$ is the size of the second largest neighboring connected component of node i, ϵ is a very positive number. As such, this measure essentially prefers to recover the node with fewer neighboring connected components. If more than one node have the same minimum $\mathcal{N}_i$, the node with the smallest $\mathcal{M}_{i,2}$ will be selected.

Previous work [9, 28] shows that the latter measure performs the best for reinsertion. The author of [28] uses a max-heap data structure to efficiently scale it to very large graphs.

3.2 Training algorithms

The Q score computations depend on a collection of 6 parameters $\Theta = \{\theta_i\}_{i=1}^6$. We use Q-learning [18, 27] to update them. More specifically, Q-learning updates the parameters by performing gradient descents on samples (or mini-batches) of experience (s, a, r, s'), to minimize the loss defined as:

$$\text{Loss} = \mathrm{E}_{(s,a,r,s')\sim U(D)}[(r + \gamma \max_{a'} \hat{Q}(s', a') - Q(s_t, a_t))^2] \tag{5}$$

Here the samples (s, a, r, s') are drawn uniformly at random from the pool of the stored samples $U(D)$, and $D = \{e_1, e_2, ..., e_t\}, e_t = (s_t, a_t, r_t, s_{t+1})$. $\hat{Q}$ is the target network, of which the parameters are only updated with the Q network parameters every C steps, and are held fixed between individual updates [27].

To deal with the delayed rewards, we wait n steps before updating parameters, so as to collect a more accurate estimate of the future rewards [21]. Then the sampled experience should be $(s_t, a_t, r_{t,t+n}, s_{t+n})$, where $r_{t,t+n} = \sum_{i=0}^{n} r_{t+i}$. And in our problem, the reward r_t at each time step t is set to be -1, since optimizing the Eq. 5 is to maximize the cumulative rewards, while we seek to minimize the number of removed nodes.

In the training phase, we use Barabási-Albert (BA) model [4] to generate synthetic graphs for training and validation. First, we introduce some terminology: an **episode** is the entire node removing process on a graph until the terminal state, a **trajectory** produced in the episode is a state action sequence $(s0, a0, r0, s1, r1, s2, ..., s_{terminal})$, **PlayGame** means to complete the episode on this graph, extract the $(s_t, a_t, r_{t,t+n}, s_{t+n})$ transitions from the trajectory and then store them into the replay memory buffer. During training, we use the $\epsilon-$greedy policy with ϵ linearly annealing from 1.0 to 0.05 over the 10000 episodes to balance between exploration and exploitation [33]. In the inference phase, we greedily remove the highest Q score node on the 2-core, until the game is over (here refers to 2-core becomes empty). We train the model for a total 100,000 episodes, and use a replay memory of size 50,000 to store the most recent transitions. Every 300 episodes, we evaluate the agent on 100 synthetic graphs with the same size as the training graphs, and average the performances. After each episode, we randomly sample a mini-batch of transitions from the replay memory buffer and perform the stochastic gradient descent updates to minimize the loss function (Eq. 5). Algorithm 2 describes the whole training procedure in detail.

Algorithm 2 Training Procedure of $CoreGQN$

Initialize experience replay memory D with size M
Initialize state-action pair value function Q with random weights Θ
Initialize target Q function with weights $\hat{\Theta} = \Theta$
for $episode = 1$ to N **do**
  Generate a graph G from the BA model
  Get the 2-core $\mathrm{G_{core}}$ from G
  Initialize the state to an empty sequence $S_1 = ()$
  for $t = 1$ to T **do**
    With probability ε, select a random action a_t on $\mathrm{G_{core}}$
    Otherwise select $a_t = argmax_a Q(S_t, a; \Theta)$
    Execute action a_t (remove node a_t from $\mathrm{G_{core}}$) and observe reward r_t
    Add a_t to partial solution $S_{t+1} = S_t \cup a_t$
    if $t \geq n$ **then**
      Store transition $(S_{t-n}, a_{t-n}, r_{t-n,t}, S_t)$ in D
      Sample random mini-batch of transitions $(S_j, a_j, r_j, S_j^{'})$ from D
      Set $y_j = \begin{cases} r_j, & \text{if episode terminates at step j+1} \\ r_j + \gamma max_{a'} Q(S_j^{'}, a^{'}; \hat{\Theta}), & \text{otherwise} \end{cases}$
      Perform a stochastic gradient descent over (5) with respect to the network parameters Θ
      Every C steps reset $\hat{\Theta} = \Theta$
      Update $\mathrm{G_{core}}$ with the 2-core in the remaining graph
    end if
  end for
end for

3.3 Complexity analysis

We use sparse matrix to represent the real-world networks. Time complexity in the decycling stage is $O(T|E|t)$, where T is the terminating iterations for graph embedding, $|E|$ is the number of edges, and t is the number of greedy steps (in the worst case, approximates the number of nodes N). In our experiments, we found the performance is almost unaffected if we remove a finite fraction (α, up to 1%) of nodes at each adaptive step, then we have $t = 1/\alpha$, which is a constant that is dependent of graph sizes. As a result, our decycling stage takes time that is linear to the number of edges.

For the tree breaking algorithm, it can be implemented in time $O(N(\log N + D))$ [7], where D is the maximal diameter of the trees inside the remaining forest. The computational cost of reinserting a node in the graph is entirely bounded by $k_{\max} C' \log(k_{\max} C')$ [7], where $k_{\max}$ is the maximal degree of the graph, C' is the target limit size. In sparse graphs, the update cost is typically sub-linear in N, making the reinsert strategy very efficient.

Note that compared to the current three-stage algorithms, MinSum [7], BPD [29] and CoreHD [35], CoreGQN mainly differs in the decycling part, and is superior to these methods in the decycling complexity.

4 Experiments

In this section, we demonstrate the effectiveness and efficiency of CoreGQN on real-world networks. We first explain experimental settings in detail, including baselines and datasets. Then discuss the results.

4.1 Experimental setup

4.1.1 *Baseline methods*

Collective Influence(CI) [28]. The Collective Influence measure is defined as the product of the node's reduced degree (i.e. original degree minus 1) with the sum of the reduced degrees of the nodes that are within a constant hops away from it. This measure describes the proportion of other nodes that can be reached from a given node, assuming the nodes with higher CI values play more crucial roles in networks. The CI method sequentially removes the node with the highest CI value and recalculating the collective influence for the rest following operations.

MinSum [7]. MinSum is proposed to address the network dismantling problem. It consists three stages, which firstly utilizes a variant of message-passing algorithm to break all the cycles, and then breaks the remaining tree into small

components by removing a fraction of nodes that vanishes in the large size limit. In the third stage, it greedily reinserts some nodes that close cycles without increasing too much the size the largest component, to reduce the total number of nodes removed. Since another algorithm BPD. [29] performs very close to MinSum, we do not compare with BPD in this paper.

CoreHD [35]. CoreHD recursively remove nodes of the highest degree from the 2-core of the network. It is based on the observation that the network contains no loops if and only if the 2-core is empty. After breaking all the cycles, it adopts the similar tree-breaking algorithm and reinsertion technique to obtain the final solution.

GND [31]. GND is the state-of-the-art method to address the network dismantling problem with non-unit removal costs. Its unit-cost version is similar to the traditional spectral cut method, which uses the least nodes to break the network into two components with equal sizes, repeat the procedure until the remaining largest component does not exceed the target limit size.

4.1.2 *Datasets*

For real-world data, we choose nine real-world networks with different scales, which covers a wide range of types. Specifically, they are:

Crime [22], a malicious network from the projection of a bipartite network of persons and crimes, each node denotes a person, an edge represents that two persons are involved in the same crime;

HI-II-14 [1], the corresponding **H**uman **I**nteractome dataset covering Space **II** and reported in 20**14**. Each node represents a distinct protein, each edge denotes the interaction between the corresponding proteins;

Digg [22], a reply network of the social news website Digg. Each node in the network is a user of the website, and each edge denotes that a user replied to another user;

Enron [24], the Enron email communication network which covers all the email communication within a dataset of around half million emails. Each node is an email address, and an edge denotes at least one email communication;

Gnutella31 [23], a sequence of snapshots of the Gnutella peer-to-peer file sharing network from August 2002. Nodes represent hosts in the Gnutella network topology and edges represent connections between the Gnutella hosts;

Epinion [22], the trust network from the online social network Epinions. Nodes are users of Epinions and directed edges represent trust between the users;

Facebook [22], contains friendship data of a small subset of Facebook users. A node represents a user and an edge represents a friendship between two users;

Youtube [22], the friendship network of the video-sharing site Youtube. Nodes are users and an edge between two nodes indicates a friendship;

Flickr [22], the social network of Flickr users and their connections.

We treat all the networks as undirected ones and remove the self-loops. We extract the largest connected component from them as the test datasets. Basic statistics of the extracted networks are reported as Table 1.

4.1.3 Evaluation metrics

For all baseline methods and CoreGQN, we report their effectiveness in terms of the percolation fraction, and their efficiency in terms of wall-clock running time.

Percolation Fraction is defined as the minimum fraction of nodes we need to remove in order to break the network into components with size smaller than the target limit size.

$$q_c = \min\{q \in [0,1] : s(q) \leq \text{threshold}\} \tag{6}$$

where q is the fraction of removed nodes, $s(q)$ is the fraction of nodes in the remaining largest component. *threshold* is target limit fraction, here we set it as 1%.

Wall-clock running time is defined as the actual time taken from the start of a computer program to the end, usually in seconds.

4.1.4 *Other settings*

All experiments are conducted on a 20-core server with 512GB memory and a 16 GB Tesla V100 GPU. We generate small scale (30–50 nodes) BA graphs as the training sets and validation sets. We randomly generate 20,000 synthetic networks for training and 100 ones for validation. The model is implemented with Tensorflow 1.8, and is trained with Adam optimizer. Hyper-parameters (Table 2) are tuned according to the performance on the validation set. The training process terminates with early stopping based on the validation set. Empirically, the model converges very nicely (Fig. 2) on this problem.

For the baselines implementations, we use the source codes [2],[3],[4],[5],[6] released online, and use the default parameter settings for each method.

[1] http://interactome.baderlab.org/data/Rolland-Vidal(Cell2014).psi.

[2] https://github.com/zhfkt/ComplexCi.

[3] http://power.itp.ac.cn/zhouhj/codes.html.

[4] https://github.com/abraunst/decycler.

[5] https://github.com/hcmidt/corehd.

[6] https://github.com/renxiaolong/Generalized-Network-Dismantling.

Table 1 Basic statistics for real-world networks

Statistics	Malicious	PPI	Communication		Infrastructure	Social			
	Crime	HI-II-14	Digg	Enron	Gnutella31	Epinions	Facebook	Youtube	Flickr
Nodes num	829	4165	29,652	33,696	62,561	75,877	63,392	1,134,890	1,624,991
Edges num	1473	13,087	84,781	180,811	147,878	405,739	816,831	2,987,624	15,473,043
Max degree	25	286	310	1383	95	3044	1098	28,754	27,224
Avg degree	3.55	6.28	5.72	10.73	4.73	10.69	25.77	5.27	19.04
Diameter	10	11	5.72	11	11	15	15	24	24
Mean shortest length	5.04	4.16	4.68	4.03	5.96	4.40	4.31	5.55	5.19
Clustering coefficient	0.0058	0.0444	0.0054	0.5092	0.0055	0.1378	0.2218	0.0808	0.1892
Assortativity	− 0.1645	− 0.2016	0.0027	− 0.1165	− 0.0927	− 0.0406	0.1768	− 0.0369	− 0.0167
Powerlaw exponent	2.65	2.68	2.46	2.62	2.91	2.69	2.92	2.49	2.63

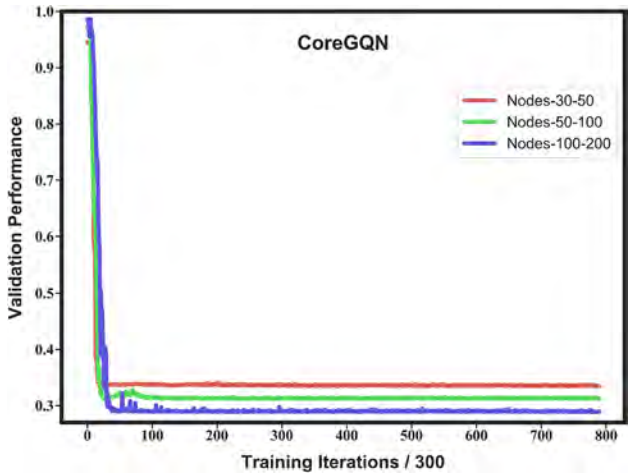

Fig. 2 CoreGQN convergence measured by the validation performance. We train an agent to decycle the network on synthetic BA graphs, we train it on different scales (more specifically, with node set size containing 3–50, 50–100 and 100–200) and valid it on the same scale. For each iteration, we sample 64 synthetic graphs to train and update the parameters. Every 300 iterations, we valid the current model on 100 randomly generated graphs, and the performance is measured by the fraction of nodes that are needed to break all the loops in the graph. We can see for different training scales, our CoreGQN could always converge very quickly on the validation dataset

4.2 Results

We evaluate CoreGQN on nine real-world networks from the perspective of both effectiveness (Table 3) and efficiency (Table 4). These networks cover different types, including malicious networks, metabolic networks, communication networks, infrastructure networks and social networks, which represents a wide range of practical applications for network dismantling. Besides, the scale of these networks ranges from thousands to millions, meeting the needs for the majority of realistic problems. We compare against the most representative methods for the network dismantling problem. All these contribute to the following fair and convincing comparisons.

We use the agent trained from BA graphs with small scales, to break all the loops in the networks at first; then adopt the tree-breaking algorithm (3.1) to dismantle the remaining forests into sub-extensive components such that the largest one is smaller than $0.01N$; after that, we add back some nodes without increasing the largest size. We report the size of the final solution as the percolation fraction result in Table 3, and the running time (exclude the time for reading graph) in Table 4. Note that for CoreGQN's time result, we do not include the training time. Since other methods do not need to train, it is sometimes unfair for the comparison. However, considering that CoreGQN only needs to be trained once off-line and can then generalize to any unseen network, it is reasonable not to consider the training time in comparison. Actually, as shown in Fig. 2, CoreGQN can converge very soon, resulting in an acceptable training time, e.g., it takes about 1 hours to train and select the model used in this paper.

We can see from Table 3 that CoreGQN works excellently on real-world networks, giving dismantling sets very close to the close-to-optimal state-of-the-art MinSum algorithm, and much better than other methods, such as CI and GND. In some networks, like HI-II-14 and Epinions, we even beat MinSum slightly. In terms of efficiency, CoreGQN is also superior, especially on very large network, e.g., on Flickr network with millions of nodes and tens of millions of edges,

Table 2 List of hyper-parameters and their values

Hyperparameter	Value	Description
Replay memory size	500,000	Capacity of experience replay buffer
Learning rate	0.0001	The learning rate used by Adam optimizer
Embedding dimension	64	Dimension of node embedding vector
Update time	1000	The frequency with which target network is updated
Layer iterations	5	Number of neighbor-aggregation iterations
Maximum episodes	1,000,000	Maximum episodes for the training process
Discount factor	1	Discount factor gamma used in Q-learning update
Q-learning steps	5	Number of steps for multi-step Q-learning algorithm
Mini-batch size	64	Number of mini-batch training samples

All parameters are obtained by an informal search, and due to the high computational cost, we did not perform a systematic grid search, we believe that the experimental results will be improved by systematically tuning the hyper-parameter values

Table 3 Comparative results of percolation fraction (%) on real-world networks

Method	Crime	HI-II-14	Digg	Enron	Gnutella31	Epinions	Facebook	Youtube	Flickr
CI	55.73	9.96	13.50	10.94	16.98	9.08	37.46	4.90	11.26
MinSum	19.30	9.34	11.77	7.86	14.92	7.82	35.42	4.01	7.57
CoreHD	19.54	9.65	12.01	13.60	15.35	10.32	38.38	4.94	10.31
GND	25.57	13.42	18.41	11.57	20.73	10.79	47.89	5.62	12.04
CoreGQN	19.66	9.34	11.93	7.95	15.28	7.81	36.61	4.05	8.28

The results of CoreGQN are achieved by removing 1% of the total nodes on current 2-core at each adaptive step

Table 4 Comparative results of running time (/s) on real-world networks

Method	Crime	HI-II-14	Digg	Enron	Gnutella31	Epinions	Facebook	Youtube	Flickr
CI	0.01	0.49	10.99	70.74	9.38	668.43	2723.23	7056.41	1,382,460.00
MinSum	2.02	20.04	134.49	323.55	225.42	681.12	1407.99	5478.19	35,215.20
CoreHD	0.06	0.89	7.19	25.76	8.63	95.81	146.63	1194.03	27928.84
GND	0.11	1.32	55.01	35.70	386.31	233.14	4614.54	52246.20	815,411.00
CoreGQN	0.23	1.78	10.29	22.52	31.57	38.90	200.37	410.82	2914.54

The results of CoreGQN are achieved by removing 1% of the total nodes on current 2-core at each adaptive step

we are over 10 times faster than the best performed MinSum. Taking Tables 3 and 4 together, CoreGQN achieves a better balance between effectiveness and efficiency, than the most competing methods, suggesting a new promising option for practical network dismantling problems.

We also show the robustness curves for all the methods on these networks in Fig. 3. The curve is plotted with the horizontal axis being the fraction of removed nodes, and the vertical axis being the largest (giant) component size in the remaining graph. The curves again show that CoreGQN performs very close to MinSum, and much better than the other baselines, in terms of the percolation point where the GCC size decreases to 1%. We also observe that although CI or GND has a much worse performance for the percolation fraction, the curves they generated are lower than MinSum and ours, indicating they have a better ability in capturing the network's response to the dismantling throughout the whole process, rather not just the percolation point, which is as well meaningful in practice [32]. How to minimize both the percolation fraction and the area under the robustness curve remains a future topic in this direction.

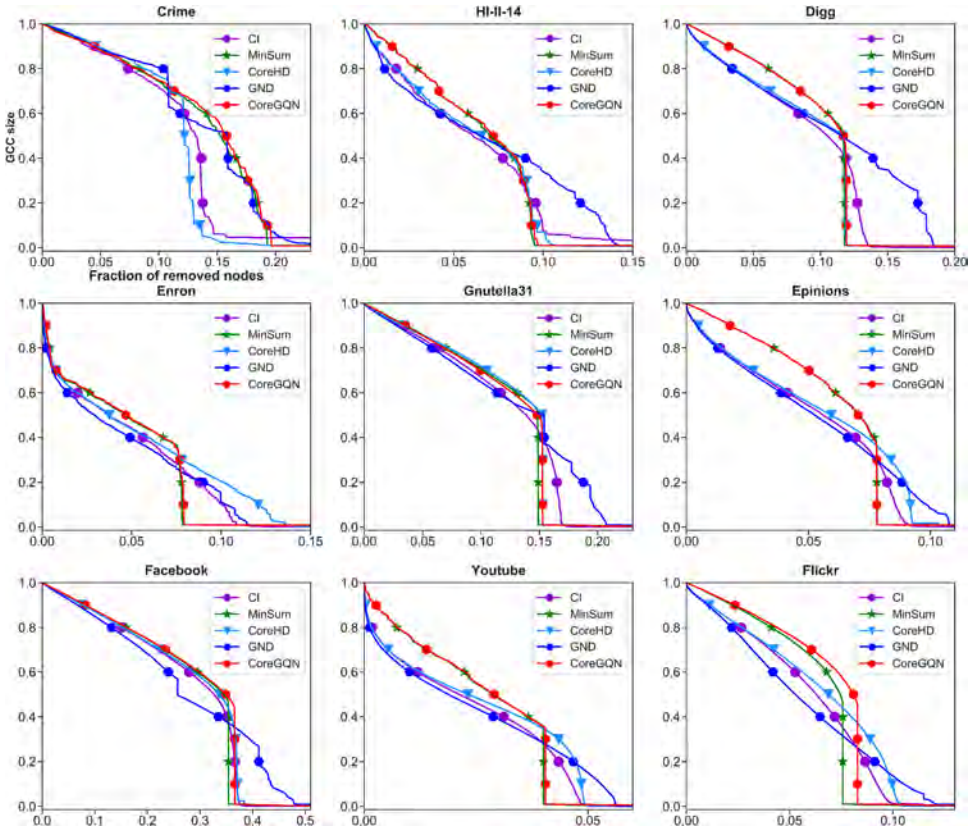

Fig. 3 Comparative results of robustness curves on real-world networks. The horizontal axis is the fraction of the removed nodes, and the vertical axis is the remaining giant connected component (GCC) size

5 Conclusion

In this paper, we purposed a learning-based framework for efficiently dismantling large networks. We combined the graph embedding and reinforcement learning to train an agent, which learns a smart strategy to decycle the graph effectively through many self-plays on toy networks, e.g., Barabási-Albert graphs. This decycling strategy learned from small synthetic graphs was able to generalize well on very large real-world networks. After removing all the loops with the learned agent, we broke the remaining tree components to reach the limit size, and reinsert some nodes without increasing the largest size to reduce the total number of the removed nodes. Extensive comparisons on real-world data against the state-of-the-art baselines showed that CoreGQN could achieve the better trade-off between effectiveness and efficiency. We believe CoreGQN opens a new perspective to understand the robustness of the complex systems, and there remains more promising explorations in the further. We will later release the code and trained models to support future progress along this direction.

References

1. Albert R, Barabási AL (2002) Statistical mechanics of complex networks. Rev Mod Phys 74(1):47
2. Altarelli F, Braunstein A, Dall'Asta L, Zecchina R (2013) Large deviations of cascade processes on graphs. Phys Rev E 87(6):062,115
3. Altarelli F, Braunstein A, Dall'Asta L, Zecchina R (2013) Optimizing spread dynamics on graphs by message passing. J Stat Mech Theory Exp 2013(09):P09,011
4. Barabási AL, Albert R (1999) Emergence of scaling in random networks. Science 286(5439):509–512
5. Bavelas A (1950) Communication patterns in task-oriented groups. J Acoust Soc Am 22(6):725–730
6. Bello I, Pham H, Le QV, Norouzi M, Bengio S (2016) Neural combinatorial optimization with reinforcement learning. arXiv :1611.09940
7. Braunstein A, Dall'Asta L, Semerjian G, Zdeborová L (2016) Network dismantling. Proc Natl Acad Sci 113(44):12368–12373
8. Brin S, Page L (1998) The anatomy of a large-scale hypertextual web search engine. Comput Netw ISDN Syst 30(1–7):107–117
9. Clusella P, Grassberger P, Pérez-Reche FJ, Politi A (2016) Immunization and targeted destruction of networks using explosive percolation. Phys Rev Lett 117(20):208,301
10. Cohen R, Erez K, Ben-Avraham D, Havlin S (2001) Breakdown of the internet under intentional attack. Phys Rev Lett 86(16):3682
11. Dai H, Dai B, Song L (2016) Discriminative embeddings of latent variable models for structured data. In: International conference on machine learning, pp 2702–2711

12. Fan C, Liu Z, Lu X, Xiu B, Chen Q (2017) An efficient link prediction index for complex military organization. Physica A Stat Mech Appl 469:572–587
13. Fan C, Sun Y, Zeng L, Liu YY, Chen M, Liu Z (2019) Dismantle large networks through deep reinforcement learning. In: ICLR representation learning on graphs and manifolds workshop
14. Fan C, Xiao K, Xiu B, Lv G (2014) A fuzzy clustering algorithm to detect criminals without prior information. In: Proceedings of the 2014 IEEE/ACM international conference on advances in social networks analysis and mining, pp 238–243. IEEE Press
15. Fan C, Zeng L, Ding Y, Chen M, Sun Y, Liu Z (2019) Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach. arXiv:1905.10418
16. Freeman LC (1977) A set of measures of centrality based on betweenness. Sociometry 40:35–41
17. Hamilton W, Ying Z, Leskovec J (2017) Inductive representation learning on large graphs. In: Advances in neural information processing systems, pp 1024–1034
18. Hessel M, Modayil J, Van Hasselt H, Schaul T, Ostrovski G, Dabney W, Horgan D, Piot B, Azar M, Silver D (2018) Rainbow: combining improvements in deep reinforcement learning. In: Thirty-Second AAAI conference on artificial intelligence
19. Janson S, Thomason A (2008) Dismantling sparse random graphs. Comb Probab Comput 17(2):259–264
20. Kempe D, Kleinberg J, Tardos É (2003) Maximizing the spread of influence through a social network. In: Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining, pp 137–146. ACM
21. Khalil E, Dai H, Zhang Y, Dilkina B, Song L (2017) Learning combinatorial optimization algorithms over graphs. In: Advances in neural information processing systems, pp 6348–6358
22. Kunegis J (2013) Konect: the koblenz network collection. In: Proceedings of the 22nd international conference on world wide web, pp 1343–1350. ACM
23. Leskovec J, Kleinberg J, Faloutsos C (2007) Graph evolution: densification and shrinking diameters. ACM Trans Knowl Discov Data 1(1):2
24. Leskovec J, Lang KJ, Dasgupta A, Mahoney MW (2009) Community structure in large networks: natural cluster sizes and the absence of large well-defined clusters. Internet Math 6(1):29–123
25. Li Z, Chen Q, Koltun V (2018) Combinatorial optimization with graph convolutional networks and guided tree search. In: Advances in neural information processing systems, pp 539–548
26. Ma Z, Li M, Wang Y (2019) Pan: Path integral based convolution for deep graph neural networks
27. Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, Graves A, Riedmiller M, Fidjeland AK, Ostrovski G et al (2015) Human-level control through deep reinforcement learning. Nature 518(7540):529
28. Morone F, Makse HA (2015) Influence maximization in complex networks through optimal percolation. Nature 524(7563):65
29. Mugisha S, Zhou HJ (2016) Identifying optimal targets of network attack by belief propagation. Phys Rev E 94(1):012,305
30. Pastor-Satorras R, Vespignani A (2001) Epidemic spreading in scale-free networks. Phys Rev Lett 86(14):3200
31. Ren XL, Gleinig N, Helbing D, Antulov-Fantulin N (2019) Generalized network dismantling. In: Proceedings of the national academy of sciences, p 201806108
32. Schneider CM, Moreira AA, Andrade JS, Havlin S, Herrmann HJ (2011) Mitigation of malicious attacks on networks. Proc Natl Acad Sci 108(10):3838–3841
33. Sutton RS, Barto AG (2018) Reinforcement learning: an introduction. MIT Press, Cambridge
34. Velickovic P, Cucurull G, Casanova A, Romero A, Lio P, Bengio Y (2018) Graph attention networks. In: ICLR
35. Zdeborová L, Zhang P, Zhou HJ (2016) Fast and simple decycling and dismantling of networks. Sci Rep 6:37,954
36. Zhou HJ (2013) Spin glass approach to the feedback vertex set problem. Eur Phys J B 86(11):455